南华大学

计算机学院

实 验 报 告

（ 2026 年秋季学期 ）

课程名称　Web 全栈开发

姓　　名　王亨泰

学　　号　20244081108

专　　业　数据科学与大数据技术（中国电子新创班）

2026 年 10 月

CONTENTS

目 录

01 实验概述 1

02 开发环境与技术选型 3

03 需求分析与总体设计 5

04 Vue 3 知识点覆盖与实现 9

05 核心页面与功能实现 13

06 关键代码解析 21

07 测试与运行结果 26

08 部署与公网访问 29

09 问题与解决 32

10 总结与体会 34

A **附录 A：27 个单文件组件清单** 36

B **附录 B：参考文献** 38

报告中的页面截图均来自项目实际运行结果；公网演示地址与有效期见第 8 章。

CHAPTER 01 / OVERVIEW

01 实验概述

1.1 实验背景与目的

本次实验以 Vue 官方教程为学习主线，要求在掌握模板语法、响应式、组件化、路由、表单与过渡等知识的基础上，独立完成一个只有前端、至少包含 3 个页面和 15 个单文件组件（SFC）的网站。实验重点不是堆叠功能，而是把教程中的零散知识点组织成一个可以真实浏览、可交互、可部署的完整作品。

我选择“独立书店”作为主题，完成了「纸间书店 / Paper Between」。网站以暖纸色、森林绿和陶土橙为主色，用宋体标题、纸张纹理、手绘感封面和留白营造安静的阅读氛围。功能上包含书目浏览、搜索筛选、加入书架、阅读进度、夜间模式、异步读者短评、活动报名、FAQ 和联系表单，基本覆盖了一个内容型网站从前台展示到轻量交互的主要环节。

实验的目标可以概括为三点：第一，用组件化思维拆分页面，避免把逻辑堆在单个文件中；第二，把 Vue 教程中的响应式、组件通信、路由、过渡与组合式函数真正用起来；第三，让网站可以在普通静态托管上直接访问，并形成可复现的构建、测试与部署流程。

1.2 实验任务

- **页面要求：**至少 3 个页面，实际完成 4 个主页面与 1 个自定义 404 页面。
- **组件要求：**至少 15 个 Vue 单文件组件，实际完成 27 个。
- **知识覆盖：**覆盖 Vue 教程中的模板语法、响应式、计算属性、侦听器、生命周期、模板引用、组件通信、插槽、路由、表单、过渡、Teleport、KeepAlive、Suspense、自定义指令等。
- **工程要求：**使用 Vite 构建，项目可安装、可启动、可打包，代码结构清晰。

- **交付要求：**提供公网可访问地址，并按照模板完成实验报告，附运行截图。

1.3 完成情况

项目	完成情况	说明
页面数量	4 个主页面 + 404	首页、书架、书籍详情、关于、缺省页
SFC 数量	27 个	5 个页面 + 21 个组件 + App.vue
构建结果	通过	Vite production build 无错误
自动化验收	通过	12 项交互检查，无 console / pageerror / requestfailed
公网访问	已上线	BrewPage 静态托管，30 天有效
实验报告	本文件	含设计、代码、截图、测试与部署说明

表 1-1　实验完成情况

> **作品一句话**
>
> 一家不着急的独立书店：选书、读书、在纸页之间慢下来。网站用 Vue 3 把“浏览—筛选—收藏—阅读—交流”串成了一条完整但轻盈的路径。

CHAPTER 02 / ENVIRONMENT

02 开发环境与技术选型

2.1 开发环境

项目在 Windows 11 环境下完成，使用 DeepSeek Harness 提供的 Node.js 与 pnpm 运行时。依赖安装、构建、预览和浏览器验收都在项目目录中完成，所有命令均可复现。

类别	版本 / 工具	用途
操作系统	Windows 11	开发与截图环境
Node.js	v24.21.0	运行 Vite、脚本与工具
包管理器	pnpm 11.28.5	依赖安装与锁版本
构建工具	Vite 6.4.5	开发服务器与生产构建
框架	Vue 3.5.44	页面与组件
路由	Vue Router 4.6.4	单页应用路由
浏览器	Microsoft Edge 148	自动化验收与截图
测试工具	Playwright Core 1.64.1	无头浏览器交互测试
报告工具	python-docx 1.2.0	生成实验报告

表 2-1 开发环境与工具

2.2 技术选型

框架选择 Vue 3 的 Composition API 与 `<script setup>` 语法。相比 Options API，组合式写法更适合本项目这种“多个页面共享书架状态、多个组件共享 Toast、搜索与筛选逻辑较集中”的场景。所有页面和组件都使用单文件组件，模板、逻辑与局部样式放在同一个文件中，便于定位和复用。

构建工具选择 Vite，原因是启动快、配置少，并且可以直接把 `dist/` 作为静态

站点部署。路由选择 Vue Router 4 的 Hash 模式：静态托管不需要额外的服务器重写规则，刷新 `/#/books/siddhartha` 这类地址也不会 404。状态管理没有引入 Pinia，而是用组合式函数 `useShelf()` 配合 `provide/inject` 与 localStorage。这样既覆盖了教程中的依赖注入与侦听器，也让项目依赖保持精简。

样式部分没有使用 UI 组件库。页面全部用 CSS 变量、Grid、Flex、Container Queries 和少量内联 SVG 完成。书籍封面不是图片，而是 `BookCover.vue` 根据每本书的 `palette` 与 `motif` 动态渲染的 CSS 封面；书店门面、地图、空状态插画也用 CSS 绘制。这保证了离线可用、加载稳定，也让视觉风格更统一。

2.3 项目运行

```
pnpm install
pnpm dev          # http://127.0.0.1:5173
pnpm build        # 生产构建，输出到 dist/
pnpm preview      # http://127.0.0.1:4173

# 自动化验收与截图
node scripts/verify.mjs
node scripts/capture.mjs
```

代码 2-1　项目启动、构建与验收命令

CHAPTER 03 / DESIGN

03 需求分析与总体设计

3.1 主题定位与视觉系统

“纸间”被设定为一家开在衡阳常胜西路的独立书店：不追求畅销榜，不催促消费，允许读者在窗边坐一个下午。这个设定决定了网站的气质——不是电商式的强转化，而是内容式的慢浏览。因此页面大量使用留白、大字号宋体标题、纸张底色和低饱和插画，交互反馈也尽量轻，避免弹窗和强刺激。

视觉系统以“纸、墨、光”为线索：纸白是背景，森林绿是书架与主按钮，陶土橙用于强调，旧纸金用于评分、印章和分隔。标题使用宋体 / Songti 字族，正文使用无衬线中文字体，英文与数字使用等宽字体作为节奏点。夜间模式把纸白转为深墨绿，保留暖金色作为光源，形成“夜读”氛围。

角色	色值	使用位置
纸白	#F4EDE1	页面背景、卡片底色
森林绿	#2F4A3D	主按钮、深色区块、页脚
陶土橙	#BD6748	强调文字、印章、进度
旧纸金	#D3A34B	评分、装饰线、夜间光源
墨色	#20332A	正文与标题
浅纸	#FAF5EC	工具栏、表单、次级卡片

表 3-1 视觉系统的主要颜色

3.2 功能需求

模块	需求	实现方式
首页	品牌展示、精选书目、主题书架、共读、活动、订阅	Hero + SectionTitle + BookCard + QuoteMarquee + NewsletterForm
书架	搜索、分类/气质筛选、排序、网格/列表切换、空状态	SearchField + FilterPills + computed + TransitionGroup
详情	动态路由、Tabs、异步短评、阅读进度、关联推荐	useRoute + BookTabs + Suspense + ReadingProgress
书架状态	加入/移出、数量统计、进度、刷新后保留	useShelf + provide/inject + localStorage
主题	日间/夜间切换、记忆偏好	useTheme + watchEffect + CSS Variables
关于	故事、时间线、团队、活动、FAQ、联系表单	FaqList + ContactForm + 表单校验
响应式	桌面、平板、手机三档布局	CSS Grid + 媒体查询 + 移动菜单
可访问性	键盘可操作、语义标签、减少动效	aria-* + focus-visible + prefers-reduced-motion

表 3-2 功能需求与实现方式

3.3 页面与路由设计

路由	页面组件	关键能力
/	HomeView	品牌首页、精选、主题书架、共读、活动、订阅
/shelf	ShelfView	搜索、筛选、排序、视图切换、URL query 同步
/books/:id	BookDetailView	动态路由、beforeEnter 校验、Tabs、异步短评
/about	AboutView	故事、时间线、团队、FAQ、表单、地图
/:pathMatch(.*)*	NotFoundView	自定义 404

表 3-3 页面与路由设计

路由使用 `createWebHashHistory()`，无论部署在根目录还是子目录，都能保持页

面刷新可用。`/books/:id` 通过 `props: true` 把参数传给详情页，并设置 `beforeEnter` 守卫：如果 id 不存在，直接跳转到 404，而不是在页面里渲染半截内容。`scrollBehavior` 负责在路由切换时回到顶部，并在带 hash 的链接中平滑滚动到锚点。

3.4 组件化设计

项目按“页面—业务组件—基础组件”三层拆分。页面只负责组织区块和调用组合式函数；业务组件处理书籍卡片、封面、短评、阅读进度等具体语义；基础组件只提供按钮、标签、标题、空状态等通用能力。这样的拆分让 27 个 SFC 都有明确职责，而不是为了数量而拆分。

```
App.vue
├─ AppHeader.vue
│  └─ ThemeToggle（通过 provide 注入的 theme）
├─ RouterView + KeepAlive + Transition
│  ├─ HomeView.vue
│  │  ├─ BookCard.vue → BookCover.vue / StarRating.vue
│  │  ├─ SectionTitle.vue / BaseButton.vue / BaseTag.vue
│  │  ├─ QuoteMarquee.vue / StatBlock.vue
│  │  └─ NewsletterForm.vue
│  ├─ ShelfView.vue
│  │  ├─ SearchField.vue / FilterPills.vue
│  │  ├─ BookCard.vue / EmptyState.vue
│  │  └─ TransitionGroup + v-memo
│  ├─ BookDetailView.vue
│  │  ├─ BookTabs.vue / ReadingProgress.vue
│  │  └─ Suspense → BookReviews.vue
│  └─ AboutView.vue
│     ├─ FaqList.vue / ContactForm.vue
│     └─ NewsletterForm.vue
├─ ShelfDrawer.vue（Teleport）
├─ ToastStack.vue（Teleport + TransitionGroup）
└─ AppFooter.vue（v-once）
```

代码 3-1　组件层级示意

3.5 数据流与状态管理

书目、主题书架、活动和语录集中放在 `src/data/books.js`，作为只读数据源。用户的“我的书架”、阅读进度、主题偏好和 Toast 属于运行时状态：书架与进度由 `useShelf()` 管理，通过 `App.vue` 的 `provide` 注入到所有后代组件；主题由 `useTheme()` 管理；Toast 由 `useToast()` 管理。这样避免了层层 props 传递，也没有为一个练习项目引入额外状态库。

`useShelf()` 内部使用 `reactive` 保存 ids 与 progress，用 `computed` 派生书架书目、数量、阅读中数量、读完数量和平均进度，用 `watch` 深度监听状态并写入 localStorage。刷新页面后，书架和进度会自动恢复。搜索、筛选、排序属于书架页的局部状态，不进入全局 store，避免把页面细节耦合到全局。

CHAPTER 04 / VUE CONCEPTS

04 Vue 3 知识点覆盖与实现

本章不按“API 清单”罗列，而是按实际使用场景说明每个知识点解决了什么问题。所有知识点都能在项目源码中找到对应实现，完整对照见 4.8 节。

4.1 响应式、计算属性与列表渲染

书架页是本项目响应式使用最集中的地方。搜索关键词、分类、气质、排序、视图模式分别用 `ref` 保存；`filteredBooks` 用 `computed` 从 12 本书中派生结果：先按主题书架、分类、气质和关键词过滤，再按评分、年份或书名排序。由于 `computed` 会自动追踪依赖，用户修改任一筛选条件，结果和数量都会同步更新，不需要手动调用刷新函数。

列表使用 `v-for` 渲染，配合 `:key="book.id"` 保证复用正确；空结果用 `v-if` 切换到 `EmptyState`；网格与列表视图通过 `v-if / v-else` 切换。列表视图还使用了 `v-memo="[book.id, shelf.has(book.id)]"`，在书籍数据和书架状态都没变时跳过该行的重复更新，作为对列表渲染性能的一次实践。

4.2 侦听器与 watchEffect

项目中有三类典型侦听场景。第一类是“输入后延迟同步 URL”：搜索框内容变化时，用 `watch(search)` 启动 320 ms 防抖，再调用 `router.replace()` 把关键词写入 query，既避免每次按键都产生历史记录，也让搜索结果可以被刷新和分享。第二类是“状态持久化”：`useShelf()` 用深度 `watch` 监听 ids 与 progress，变化后写入 localStorage。第三类是“主题副作用”：`useTheme()` 用 `watchEffect` 自动读取 `theme.value`，同步 `<html data-theme>`、`color-scheme` 和 localStorage，不需要分别写三个 watch。

> **一个实际踩到的坑**
> 最初 `App.vue` 用 `route.fullPath` 作为 RouterView 的 key，搜索时 query 变化会重建整个书架页，导致清空搜索后旧关键词又被路由监听写回。改为 `route.path` 后，query 变化只更新状态，不再重建页面；同时保留 `/books/:id` 在切换书籍时重新挂载的能力。

4.3 生命周期与模板引用

`AppHeader` 在 `onMounted` 中注册滚动监听，在 `onUnmounted` 中移除，用来切换吸顶栏的毛玻璃背景；`QuoteMarquee` 在挂载时启动自动轮播，卸载时清理定时器；`StatBlock` 用 `IntersectionObserver` 在数字进入视口时启动 `requestAnimationFrame` 计数动画，组件卸载时取消动画与观察器。书架页被 `KeepAlive` 缓存，因此额外使用 `onActivated` 处理从页头搜索入口进入时的输入框聚焦。

模板引用用于需要直接操作 DOM 的场景：`SearchField` 通过 `ref` 拿到 input，并 `defineExpose({ focus })` 暴露聚焦方法；`ShelfDrawer` 打开后用 `nextTick` 聚焦关闭按钮；`StatBlock` 用模板引用观察自身是否进入视口。这些引用都只服务于无障碍或浏览器 API，不用来替代响应式状态。

4.4 组件通信：props、emits、v-model、插槽与依赖注入

组件通信按关系选择方式。父子之间优先 props / emits：`BookCard` 接收 `book`、`index`、`compact`，把“加入书架”的动作通过 `toggle` 事件交给页面处理；`BookTabs` 接收 tabs 数组，用 `defineModel` 与父组件双向绑定当前标签；`ReadingProgress` 用 `defineModel` 绑定 0–100 的进度，并在跨过 100 时 emit `finished`。

插槽用于让基础组件保留结构而不限制内容：`BaseButton` 提供默认插槽、`icon` 与 `trailing` 具名插槽；`SectionTitle` 提供 `title`、`description` 与 `action` 插槽；`EmptyState` 用默认插槽放操作按钮。跨层级共享则使用 `provide / inject`：

`App.vue` 提供 shelf、theme、toast 三个对象，页头、卡片、抽屉、Toast 等后代直接注入，减少了中间层透传。

4.5 路由、守卫与查询参数

路由层做了四件事：第一，使用动态路由 `/books/:id` 复用详情页；第二，使用 `beforeEnter` 校验 id，非法 id 直接进入 404；第三，使用 `scrollBehavior` 处理回到顶部、浏览器前进后退恢复位置和锚点滚动；第四，书架页把 q、category、mood、sort、view 写入 query，刷新或分享链接时能还原筛选状态。页头的搜索入口使用 `/shelf?focus=1`，书架页在挂载/激活后自动聚焦搜索框。

4.6 过渡、Teleport、KeepAlive 与 Suspense

页面切换使用 `<Transition name="page" mode="out-in">`，列表筛选使用 `<TransitionGroup>`，抽屉与移动菜单使用带位移的 Transition，Toast 使用 TransitionGroup 做进出与位移动画。移动菜单、书架抽屉和 Toast 都通过 `<Teleport to="body">` 挂到 body，避免被父级 `overflow` 或 `z-index` 裁剪。

书架页被 `<KeepAlive :include="['ShelfView']">` 缓存，返回时保留搜索和滚动位置；详情页的“读者短评”用 `defineAsyncComponent` 异步加载，并用 `<Suspense>` 展示加载占位，组件内部通过顶层 `await` 模拟网络延迟，从而真实触发 Suspense 的 fallback。

4.7 自定义指令与组合式函数

`v-reveal` 是项目自定义的全局指令：挂载时给元素添加 `reveal` 类，并用单例 `IntersectionObserver` 观察；进入视口后添加 `is-visible` 触发 CSS 过渡，卸载时停止观察。指令支持传入延迟与位移，首页卡片、区块标题和图片都通过它获得轻微的滚动入场效果；当系统开启“减少动态效果”时，指令直接显示元素。

组合式函数把可复用的状态逻辑从组件中抽离：`useShelf()` 负责书架与进度，

`useTheme()` 负责主题，`useToast()` 负责提示队列。它们都没有依赖组件实例，因此可以在任何页面或组件中调用，也方便单独测试。

4.8 知识点对照表

知识点	项目中的落点	说明
模板语法 / 指令	所有页面	v-if、v-show、v-for、v-bind、v-on、v-model
ref / reactive	useShelf、ShelfView、ContactForm	基本响应式状态与对象状态
computed	useShelf、ShelfView、BookDetailView	派生书架统计、筛选结果、关联推荐
watch	ShelfView、useShelf、ReadingProgress	搜索防抖、持久化、读完事件
watchEffect	useTheme	主题 DOM 与 localStorage 同步
生命周期	AppHeader、QuoteMarquee、StatBlock	onMounted / onUnmounted / onActivated
模板引用	SearchField、ShelfDrawer、StatBlock	聚焦、观察、DOM 引用
props / emits	BookCard、BookTabs、ReadingProgress	父子通信与事件上抛
v-model / defineModel	SearchField、FilterPills、ReadingProgress	双向绑定与组件级 model
插槽	BaseButton、SectionTitle、EmptyState	默认插槽与具名插槽
provide / inject	App.vue → 全局组件	shelf、theme、toast 跨层共享
动态组件	RouterView、BaseButton、SectionTitle	根据数据渲染不同标签/页面
KeepAlive	App.vue	缓存书架页状态
Transition / TransitionGroup	页面、抽屉、Toast、书目列表	进出场与列表位移动画
Teleport	AppHeader、ShelfDrawer、ToastStack	挂载到 body，避免层级裁剪
Suspense / 异步组件	BookDetailView → BookReviews	短评面板异步加载与 fallback
自定义指令	directives/reveal.js	IntersectionObserver 滚动入场
路由	router/index.js、ShelfView	动态路由、守卫、query、scrollBehavior
表单	ContactForm、NewsletterForm	校验、修饰符、提交状态
性能与可访问性	v-once、v-memo、aria-*、reduced-motion	减少重复更新与尊重系统偏好

表 4-1 Vue 3 知识点与项目落点对照

CHAPTER 05 / IMPLEMENTATION

05 核心页面与功能实现

5.1 首页：先建立气质，再引导浏览

首页承担“品牌说明 + 内容入口”的双重任务。首屏左侧是三条信息：地点与营业状态、主标题“让好书找到下一位读者”、一句阅读主张；右侧用三本 CSS 封面组成不对称的书堆，背后是拱门、光晕和手写便签。书堆在鼠标悬停时会轻微散开，吸引用户把视线从文字移到书上。

首屏下方是四个数字统计与滚动字幕，随后进入“编辑精选”。精选不是按销量排序，而是显示 `featured: true` 的四本书，并用 `BookCard` 统一呈现封面、评分、作者、页数和“加入书架”按钮。主题书架区使用深绿色背景，三个主题分别对应“在自然里找回呼吸”“微小而确定的幸福”“看见世界的褶皱”，点击后进入书架页并自动套用主题筛选。



图 5-1 首页首屏：品牌标题、书堆与营业信息



图 5-2　首页编辑精选：四本书与评分、加入书架入口

5.2 书架：搜索、筛选与视图切换

书架页把“找到一本书”拆成四个可组合的动作：输入关键词、选择分类、选择气质、选择排序。搜索框支持清空和 `/` 提示，分类用 `FilterPills` 组件，气质和排序用原生 select，视图用分段按钮切换。筛选条件变化后，`filteredBooks` 立即更新，结果数量显示在标题下方；如果没有结果，显示 `EmptyState` 并提供“看看全部书目”按钮。

书架页还把筛选条件同步到 URL query。这样刷新页面不会丢失筛选，复制链接给同学也能还原同样的结果。搜索输入使用 320 ms 防抖，避免每次按键都写入历史；清空搜索后会移除 q 参数。书架页被 KeepAlive 缓存，从详情页返回时仍保留原来的筛选和滚动位置。



图 5-3　书架页首屏：在架数量、借阅信息与筛选区



图 5-4　书架网格视图：封面、评分、标签与加入书架



图 5-5　书架列表视图：更适合快速扫读的密集布局

5.3 详情页：Tabs、异步短评与阅读进度

详情页通过 `/books/:id` 动态路由复用，左侧是放大的封面与拱门背景，右侧是书名、副标题、作者、评分、简介、书摘、借阅/加入书架按钮和出版信息。加入书架后，页面会出现阅读进度条，支持拖动、±10 和“标记读完”；进度写入 `useShelf()`，刷新后仍然保留。

内容区用 `BookTabs` 分为“内容简介 / 目录 / 书摘 / 读者短评”四个标签。切换标签时使用 `mode="out-in"` 的 Transition，避免内容跳动。读者短评使用异步组件 + Suspense：面板打开后先显示跳动的三个点，约 520 ms 后显示三条短评和发布表单。这个延迟不是伪造网络请求，而是用顶层 await 让 Suspense 的 fallback 真正可见。



图 5-6　书籍详情首屏：封面、书摘与借阅入口



图 5-7　读者短评：Suspense 加载完成后的评价列表与发布表单

5.4 关于页：故事、活动与表单

关于页不是简单的“简介 + 联系方式”，而是把书店的人格补全：从一张借书卡开

始的故事、三条价值观、七年的时间线、三位店员、三场活动、五个常见问题，以及一张手绘感地图。页面使用牛皮纸色与深绿色交替，避免长页面阅读疲劳。

联系表单包含姓名、邮箱、主题、留言和“希望收到回信”复选框。提交前用 computed 生成错误对象，只有全部通过才进入 750 ms 的“投递中”状态，成功后显示总结卡片并给出 Toast。表单的错误提示只在字段失焦后出现，避免用户刚打开页面就被红字包围。活动报名和 FAQ 也都有独立的交互反馈。



图 5-8 关于页首屏：书店门面 CSS 插画与七年数据



图 5-9　联系表单提交成功后的状态

5.5 响应式、夜间模式与书架抽屉

网站在 1100 / 900 / 700 / 620 / 480 px 设置断点。桌面端是左右分栏与四列书卡；平板端把书架工具栏改为上下布局；手机端把导航收进抽屉，书籍卡片改为单列，详情页封面与信息上下堆叠。移动端菜单使用 Teleport 挂到 body，打开时锁定页面滚动，点击遮罩或路由跳转后自动关闭。

夜间模式通过 `<html data-theme="night">` 切换一整套 CSS 变量，所有组件不需要写第二套样式。主题偏好写入 localStorage，刷新后保持。书架抽屉从右侧滑出，显示书架数量、平均进度、已读完数量和每本书的进度；支持移出单本、清空书架（二次确认）和跳转详情。



（a）移动端首页



（b）移动端导航菜单



（a）我的书架抽屉



（b）夜间模式首页

CHAPTER 06 / CODE REVIEW

06 关键代码解析

6.1 路由与 KeepAlive

路由表使用 Hash 历史，动态详情页通过 `props: true` 接收 id，`beforeEnter` 在进入前完成合法性检查；`scrollBehavior` 同时处理顶部、锚点和浏览器返回位置。`App.vue` 用 `KeepAlive` 缓存书架页，并用 `route.path` 作为 key，既保留 query 变化时的组件实例，又能在不同书籍之间重新挂载详情页。

```
// src/router/index.js（节选）
const router = createRouter({
  history: createWebHashHistory(),
  routes: [
    { path: '/', name: 'home', component: () => import('../views/HomeView.vue') },
    { path: '/shelf', name: 'shelf', component: () => import('../views/ShelfView.vue') },
    {
      path: '/books/:id', name: 'book', props: true,
      component: () => import('../views/BookDetailView.vue'),
      beforeEnter: (to) => getBook(String(to.params.id))
        ? true
        : { name: 'not-found', params: { pathMatch: to.path.split('/').slice(1) } },
    },
  ],
  scrollBehavior(to, from, savedPosition) {
    if (savedPosition) return savedPosition
    if (to.hash) return { el: to.hash, behavior: 'smooth', top: 110 }
    return { top: 0, behavior: 'smooth' }
  },
})
```

代码 6-1　路由、动态参数、守卫与滚动行为

```
<!-- src/App.vue（节选） -->
<RouterView v-slot="{ Component, route: currentRoute }">
  <Transition name="page" mode="out-in">
    <KeepAlive :include="['ShelfView']">
      <component :is="Component" :key="currentRoute.path" />
    </KeepAlive>
  </Transition>
</RouterView>
```

代码 6-2 KeepAlive 缓存书架页，route.path 控制重新挂载粒度

6.2 书架状态与持久化

`useShelf()` 是全局书架状态的唯一入口。ids 与 progress 用 reactive 保存，派生数据用 computed；每次变化通过深度 watch 写入 localStorage。初始化时读取并过滤无效 id，避免旧版本数据导致页面报错。

```
// src/composables/useShelf.js（节选）
const state = reactive({ ids: saved.ids, progress: saved.progress, drawerOpen: false })
const books = computed(() => state.ids.map((id) => booksById[id]).filter(Boolean))
const count = computed(() => books.value.length)
const averageProgress = computed(() => {
  if (!books.value.length) return 0
  const total = books.value.reduce((sum, book) => sum + (state.progress[book.id] ?? 0), 0)
  return Math.round(total / books.value.length)
})

watch(
  () => ({ ids: [...state.ids], progress: { ...state.progress } }),
  (value) => localStorage.setItem(STORAGE_KEY, JSON.stringify(value)),
  { deep: true },
)

function toggle(id) {
  return has(id) ? (remove(id), false) : (add(id), true)
}
```

代码 6-3 书架状态、派生统计与 localStorage 持久化

6.3 SearchField：defineModel 与模板引用

搜索框是一个受控组件：通过 `defineModel()` 与父组件的 `v-model` 双向绑定，

输入、清空、回车提交都向上同步；同时用模板引用拿到原生 input，并通过 `defineExpose` 暴露 `focus()`，让书架页在 `?focus=1` 时自动聚焦。这样既保留了组件的封装，也给父组件留了必要的控制点。

```
<!-- src/components/SearchField.vue（节选） -->
<script setup>
const model = defineModel({ type: String, default: '' })
const emit = defineEmits(['submit', 'clear'])
const inputRef = ref(null)

function clear() {
  model.value = ''
  emit('clear')
  inputRef.value?.focus()
}
defineExpose({ focus: () => inputRef.value?.focus() })
</script>

<template>
  <form role="search" @submit.prevent="emit('submit', model)">
    <input ref="inputRef" v-model="model" type="search" />
    <button v-if="model" type="button" @click="clear">清空</button>
  </form>
</template>
```

代码 6-4 defineModel、defineExpose 与模板引用

6.4 ReadingProgress：双向绑定与读完事件

阅读进度用一个 0–100 的 model 表示，父组件传入 `shelf.progressFor(book.id)`，子组件修改后通过 `update:modelValue` 写回 store。`watch` 只负责在“从小于 100 跨到 100”时触发一次 `finished`，避免每次拖动都弹提示。

```
<!-- src/components/ReadingProgress.vue（节选） -->
const model = defineModel({ type: Number, default: 0 })
const emit = defineEmits(['finished'])
const clamped = computed(() => Math.max(0, Math.min(100, Number(model.value) || 0)))

watch(
  () => clamped.value,
  (value, oldValue) => {
    if (value >= 100 && oldValue < 100) emit('finished')
  },
)

function nudge(step) {
  model.value = Math.max(0, Math.min(100, clamped.value + step))
}
```

代码 6-5　自定义 v-model、计算属性与跨阈值侦听

6.5 异步短评与 Suspense

详情页只在用户打开“读者短评”标签时才加载评论组件。`defineAsyncComponent` 负责代码分割，Suspense 负责 loading 状态。`BookReviews.vue` 顶层 `await` 520 ms，让 fallback 有真实的展示时间；加载完成后显示评论列表和发布表单。

```
<!-- src/views/BookDetailView.vue（节选） -->
const BookReviews = defineAsyncComponent(
  () => import('../components/BookReviews.vue')
)

<Suspense>
  <BookReviews :book="book" />
  <template #fallback>
    <div class="reviews-loading">
      <span class="reviews-loading__dot" />
      <p>正在把读者的便签取下来…</p>
    </div>
  </template>
</Suspense>
```

代码 6-6　异步组件与 Suspense fallback

6.6 v-reveal 自定义指令

滚动入场效果没有引入动画库，而是一个约 40 行的自定义指令。它用单例 IntersectionObserver 观察所有带 `v-reveal` 的元素，进入视口后添加 `is-visible`；系统开启“减少动态效果”时直接显示，不做动画。

```
// src/directives/reveal.js（节选）
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (!entry.isIntersecting) return
    entry.target.classList.add('is-visible')
    observer.unobserve(entry.target)
  })
}, { threshold: 0.12, rootMargin: '0px 0px -40px 0px' })

export const reveal = {
  mounted(el, binding) {
    el.classList.add('reveal')
    el.style.setProperty('--reveal-delay', `${binding.value?.delay ?? 0}ms`)
    if (matchMedia('(prefers-reduced-motion: reduce)').matches) {
      el.classList.add('is-visible')
      return
    }
    observer.observe(el)
  },
  unmounted(el) {
    observer.unobserve(el)
  },
}
```

代码 6-7　IntersectionObserver 自定义指令

CHAPTER 07 / TESTING

07 测试与运行结果

7.1 构建结果

生产构建使用 `pnpm build`，Vite 输出 85 个模块，主包 gzip 后约 63 kB，路由页面按需分包。构建过程没有报错；`dist/index.html` 使用相对路径 `./assets/...`，可直接放在根目录或子目录。

```
vite v6.4.5 building for production...
✓ 85 modules transformed.
dist/index.html                   0.71 kB | gzip: 0.49 kB
dist/assets/index-*.css          36.65 kB | gzip: 7.70 kB
dist/assets/HomeView-*.js        11.45 kB | gzip: 4.80 kB
dist/assets/ShelfView-*.js       12.65 kB | gzip: 5.04 kB
dist/assets/BookDetailView-*.js   11.27 kB | gzip: 4.85 kB
dist/assets/AboutView-*.js       18.98 kB | gzip: 7.65 kB
dist/assets/index-*.js          158.81 kB | gzip: 63.16 kB
✓ built in 1.13s
```

代码 7-1 生产构建输出（节选）

7.2 自动化验收

使用 Playwright Core 驱动本机 Edge 无头浏览器，对桌面端与移动端进行交互验收。脚本会监听 console error、pageerror 和 requestfailed，任何一项出现都会使退出码非零。测试前后使用独立的浏览器上下文，避免 localStorage 互相污染。

编号	测试项	操作	预期结果	结果
T01	首页渲染	访问 /	标题、四张精选卡出现	通过
T02	加入书架	点击第一张卡“加入书架”	徽标 +1，Toast 提示	通过
T03	书架抽屉	点击页头书架按钮	抽屉打开，显示 1 本	通过

编号	测试项	操作	预期结果	结果
T04	书架路由	点击导航“书架”	URL 为 /#/shelf，12 本书	通过
T05	搜索	输入“悉达多”	结果 1 本，URL 带 q	通过
T06	分类筛选	清空搜索后点击“生活”	结果 2 本	通过
T07	列表视图	点击列表视图按钮	渲染 12 行列表	通过
T08	详情路由	进入 /books/camellia	标题为“山茶文具店”	通过
T09	阅读进度	加入书架并拖动到 60	显示 60%，刷新后保留	通过
T10	FAQ	点击第二个问题	答案展开	通过
T11	联系表单	填写并提交	显示“信收到了”	通过
T12	404 / 移动菜单 / 夜间模式	分别访问与点击	均符合预期	通过

表 7-1　自动化验收用例

最终执行结果为：所有用例通过，控制台无错误，页面无未捕获异常，静态资源无 404。搜索、筛选、抽屉、表单等交互产生的 Toast 都能在 3.2 秒后自动消失，不会长期遮挡内容。

7.3 运行截图

桌面端截图使用 1440 × 1000 视口，移动端使用 390 × 844 视口。截图前先滚动整个页面触发 `v-reveal`，并等待字体与异步短评加载完成。以下补充展示移动端详情页与书架抽屉/夜间模式的组合效果。



图 7-1 移动端首页（390 × 844）



图 7-2 书架抽屉：数量、平均进度、单本进度与清空入口

CHAPTER 08 / DEPLOYMENT

08 部署与公网访问

8.1 构建配置

网站是纯静态单页应用，构建产物只有 HTML、CSS、JS 和 favicon。`vite.config.js` 将 `base` 设为 `./`，因此资源使用相对路径，可以部署在域名根目录，也可以部署在 `username.github.io/repo/` 这类子目录。路由使用 Hash 模式，静态服务器不需要为 `/shelf`、`/books/:id` 配置 fallback，刷新不会 404。

```
// vite.config.js
export default defineConfig({
  plugins: [vue()],
  base: process.env.VITE_BASE || './',
  server: { host: '127.0.0.1', port: 5173 },
  preview: { host: '127.0.0.1', port: 4173 },
})
```

代码 8-1　相对路径构建与本地服务器配置

8.2 公网部署与访问

最终提供两个入口：GitHub Pages 作为长期主地址，BrewPage 作为 30 天临时镜像。GitHub Pages 使用 `gh-pages` 分支托管 `dist/` 构建产物，`main` 分支保留源码、报告和脚本；BrewPage 则通过静态站点 API 上传 zip 包。两者都使用相对路径资源和 Hash 路由。

项目	内容
主地址	https://w123-123-5.github.io/paperbook/
GitHub 仓库	https://github.com/w123-123-5/paperbook
临时镜像	https://paperbook-2026-1025-8ntwatz1vs.brewpage.app/（至 2026-11-24）
发布方式	gh-pages 分支 + BrewPage 静态 zip
路由方式	Hash 路由，刷新任意页面可用
部署产物	dist/（相对路径资源）

表 8-1 公网部署信息

GitHub Pages：https://w123-123-5.github.io/paperbook/；BrewPage 镜像：https://paperbook-2026-1025-8ntwatz1vs.brewpage.app/

两个地址均使用无头浏览器再次访问：首页标题、精选卡片、书架 12 本书、详情页“我与地坛”、关于页均正常渲染，控制台无错误。GitHub Pages 走 HTTPS 并启用强制 HTTPS；BrewPage 会在页面底部注入一个轻量徽标，这是托管平台的标识，不影响站点功能。

8.3 GitHub Pages 发布流程

GitHub 的 OAuth token 缺少 `workflow` 权限，直接推送包含 GitHub Actions 工作流的 `main` 分支会被拒绝。因此这里采用更直接的 `gh-pages` 分支方案：`main` 只保存源码与报告，`gh-pages` 只保存 `dist/` 构建产物，然后在仓库 Settings → Pages 中选择 `gh-pages` 根目录。仓库中的 `docs/github-actions-deploy.yml` 留

作后续有 `workflow` 权限时切换自动部署。

```
pnpm build


# 在临时目录中准备纯构建产物
git switch --orphan gh-pages
git rm -rf .
cp -r dist/* .
touch .nojekyll
git add .
git commit -m "deploy: publish paperbook site"
git push origin gh-pages
git switch main


# 在 GitHub 仓库 Settings → Pages
# Source: Deploy from a branch
# Branch: gh-pages / (root)
```

代码 8-2 通过 gh-pages 分支发布 GitHub Pages

CHAPTER 09 / TROUBLESHOOTING

09 问题与解决

开发过程中遇到的问题主要集中在路由状态、静态部署、异步组件和视觉细节四个方面。下表记录了典型问题、原因与最终处理方式。

问题	原因	处理方式
刷新子页面 404	静态服务器没有 SPA fallback	改用 Hash 路由 + 相对路径构建
搜索清空后关键词又回来	RouterView 以 fullPath 为 key，query 变化重建页面，路由监听把旧值写回	key 改为 route.path，query 变化只更新状态
书架刷新后丢失	状态只在内存中	useShelf 深度 watch + localStorage，初始化过滤无效 id
Suspense fallback 看不到	异步组件加载太快	BookReviews 顶层 await 520 ms，保留真实 loading 阶段
封面文字溢出	长书名在固定字号下超宽	按标题长度计算 --title-size，并用容器查询单位 cqw
夜间模式局部刺眼	硬编码浅色背景	全部颜色改为 CSS 变量，按 data-theme 整体替换
移动菜单/抽屉被裁剪	父级 overflow 与 z-index 堆叠	Teleport 到 body，独立遮罩与滚动锁定
数字动画每次都重播	直接 onMounted 启动	IntersectionObserver 进入视口才启动，并只播放一次

表 9-1　典型问题与解决方式

其中“搜索清空后关键词又回来”最值得记录。表面看是搜索组件的问题，实际是

RouterView 的 key 粒度过细：每次 query 变化都销毁并重建 ShelfView，旧实例的路由监听又把关键词写回新实例。把 key 从 `fullPath` 改成 `path` 后，页面实例保持稳定，query 只作为状态来源，问题消失。这也说明 Vue 的响应式并不是“越多监听越好”，需要明确谁是单一数据源。

CHAPTER 10 / REFLECTION

10 总结与体会

10.1 完成情况

本次实验完成了一个可运行、可构建、可公网访问的 Vue 3 网站：4 个主页面 + 404、27 个单文件组件、20 余个知识点、12 本书、3 个主题书架、3 场活动、5 个 FAQ。网站支持搜索筛选、书架与进度持久化、夜间模式、异步短评、表单校验、移动端菜单和多种过渡效果，桌面与手机布局均正常。

从工程角度看，项目形成了“数据层—组合式函数—页面—组件—样式变量”的清晰分层：数据集中、状态可持久化、组件职责单一、样式可主题化。构建产物约 63 kB gzip 的主包，路由页面按需加载，静态托管即可运行。自动化验收覆盖 12 项关键交互，最终无控制台错误。

10.2 不足与改进

- **数据仍是静态的：**书目、活动和短评都来自本地数据，没有后端与真实账号体系。后续可以接入 Supabase 或自建 API，增加登录、借阅记录与真实评论。
- **搜索只做简单包含匹配：**可以加入拼音首字母、分词与高亮，提升中文检索体验。
- **无障碍还可以更细：**抽屉与移动菜单目前只处理 Esc 与焦点初始位置，完整的焦点陷阱与返回焦点仍可加强。
- **动画没有做性能分级：**低端设备可以进一步减少 blur、fixed 背景与大面积过渡。
- BrewPage 是 30 天临时镜像，GitHub Pages 是长期主地址；后续可以绑定自定义域名，并为 Pages 配置监控与缓存。

10.3 心得

这次练习最大的收获不是“记住了多少 API”，而是理解了 Vue 的响应式如何与组件边界配合。最初我把很多状态放在页面里，组件通过事件一层层往上抛；后来把书架、主题、Toast 抽成组合式函数，用 provide/inject 注入，页面和组件的职责立刻清楚了。另一个体会是：路由不是“页面切换器”，query、key、KeepAlive 和滚动行为都会影响用户对“返回”的感知，必须一起设计。

视觉上，我也第一次认真处理“留白”和“节奏”。独立书店的主题不适合高饱和、强动效，所以我把重点放在字体、纸张质感、封面母题和克制的过渡上。很多效果用 CSS 就能完成，不一定需要图片或动画库；当组件、数据和样式形成一致的语言时，网站才会像一件完整的作品，而不是知识点的拼盘。

最后，自动化验收和公网部署让这个作业从“本地能跑”变成了“可以被别人打开”。看到手机上的页面、分享出去的链接和报告里的截图来自同一份代码，是这次实验最有成就感的时刻。

CHAPTER A / APPENDIX A

A 附录 A：27 个单文件组件清单

下表按“页面、布局与基础、书籍业务、交互与表单”分组列出全部 SFC。组件数量统计不包含 `main.js` 与普通 JS 模块。

#	文件	类型	职责
1	App.vue	根组件	全局 provide、RouterView、KeepAlive、页面过渡、抽屉与 Toast 挂载
2	views/HomeView.vue	页面	首页 Hero、精选、主题书架、共读、活动、订阅
3	views/ShelfView.vue	页面	搜索、筛选、排序、网格/列表、URL query 同步
4	views/BookDetailView.vue	页面	动态详情、Tabs、异步短评、阅读进度、关联推荐
5	views/AboutView.vue	页面	故事、价值观、时间线、团队、活动、FAQ、联系表单
6	views/NotFoundView.vue	页面	自定义 404
7	components/AppHeader.vue	布局	品牌、导航、主题切换、书架入口、移动菜单
8	components/AppFooter.vue	布局	页脚品牌、营业信息、链接、回到顶部
9	components/BaseButton.vue	基础	按钮/链接多态、变体、尺寸、插槽
10	components/BaseTag.vue	基础	标签与色调
11	components/SectionTitle.vue	基础	区块标题、描述、操作插槽、对齐方式
12	components/EmptyState.vue	基础	空状态插画与操作插槽
13	components/BookCard.vue	业务	书籍卡片、评分、标签、加入书架事件
14	components/BookCover.vue	业务	CSS 动态封面、11 种母题、按标题长度自适应
15	components/StarRating.vue	业务	十分制评分转五星、半星填充
16	components/BookTabs.vue	业务	详情页标签切换、defineModel

#	文件	类型	职责
17	components/BookReviews.vue	业务	异步短评、评分表单、发布
18	components/ReadingProgress.vue	业务	阅读进度双向绑定、±10、读完事件
19	components/ShelfDrawer.vue	业务	书架抽屉、进度、移出、清空、焦点管理
20	components/SearchField.vue	交互	搜索输入、清空、提交、defineExpose 聚焦
21	components/FilterPills.vue	交互	分类筛选、defineModel
22	components/QuoteMarquee.vue	交互	语录轮播、自动播放、暂停、前后切换
23	components/StatBlock.vue	交互	数字计数动画、IntersectionObserver
24	components/NewsletterForm.vue	表单	邮箱订阅、校验、成功状态
25	components/ContactForm.vue	表单	联系表单、逐字段校验、提交状态
26	components/FaqList.vue	交互	FAQ 手风琴、aria-expanded
27	components/ToastStack.vue	交互	全局轻提示、Teleport、TransitionGroup

表 A-1　27 个单文件组件清单

CHAPTER B / APPENDIX B

B 附录 B：参考文献

[1] Vue.js 官方文档：快速上手、组件基础、响应式、路由、过渡与 KeepAlive。https://cn.vuejs.org/guide/

[2] Vue.js 官方教程（互动教程）。 https://cn.vuejs.org/tutorial/

[3] Vue Router 官方文档：动态路由、导航守卫、滚动行为。https://router.vuejs.org/zh/

[4] Vite 官方文档：静态资源、构建、base 配置。 https://cn.vite.dev/

[5] MDN Web Docs：IntersectionObserver、localStorage、CSS Container Queries。https://developer.mozilla.org/zh-CN/

[6] Playwright 官方文档：Locator、自动等待、无头浏览器。https://playwright.dev/

— 报告完 —